Modelling, Dynamics & Kinematics
Practical Assignment

Last updated: 11 November 2026

1 Introduction

The goal of this assignment is to make the end-effector of a robot arm track the full 6D pose of a moving target. You will do so for a Universal Robots UR5e arm simulated in the MuJoCo physics engine, and along the way you will build a small Python library for the geometric kinematics and control of serial-chain robots.

The differential kinematics of a serial robot is described by its geometric Jacobian. Given a coordinate frame Ψ_c, the geometric Jacobian in that frame is the linear map

$$\begin{aligned} J^c(q) \colon T_q\mathcal{Q} &\to \mathfrak{se}(3) \\ \dot{q} &\mapsto J^c(q)\,\dot{q} = T_{\mathrm{ee}}^{c,\mathrm{w}} \end{aligned} \tag{1}$$

that takes the joint velocities $\dot{q}$ to the twist $T_{\mathrm{ee}}^{c,\mathrm{w}}$ of the end-effector relative to the world, expressed in Ψ_c.

A common way to control the motion of the end-effector, for instance to track a trajectory, is to compute the desired twist of the end-effector and map it to joint velocities with the inverse of the Jacobian. This requires the Jacobian to be invertible, which is not always the case. If the robot does not have exactly six degrees of freedom, the Jacobian is not even square. Moreover, at certain configurations, called singularities, the Jacobian loses rank. Think of an arm stretched out straight that is asked to move its end-effector further in the direction of the stretch: no joint motion can achieve this, yet near such a configuration the inverse of the Jacobian returns very large joint velocities, which makes the controller unstable and the robot unsafe.

Instead of inverting the Jacobian, one can use its linear-algebraic dual,

$$\begin{aligned} \big(J^c(q)\big)^* \colon \mathfrak{se}(3)^* &\to T_q^*\mathcal{Q} \\ W^c &\mapsto W^c J^c(q) = \tau, \end{aligned} \tag{2}$$

which gives the joint torques τ whose effect on the robot is equivalent to a wrench W^c acting on the end-effector. Unlike the inverse, the dual map is defined in every configuration, singular or not, as well as for non-square Jacobians.

This suggests the following control law. From the pose $H_{\mathrm{ee}}^{\mathrm{d}}$ of the end-effector relative to the desired frame Ψ_{d}, compute the wrench W^{ee} that a virtual spring connecting the two frames would exert on the end-effector, and command the joint torques obtained from this wrench through the dual of the Jacobian J^{ee} expressed in the end-effector frame. The robot then

behaves as if its end-effector were pulled towards the target by the spring. Since a spring alone would make the end-effector oscillate around the target, a virtual damper is added, and the gravity and the other bias forces of the arm are compensated. The inverse kinematics problem is never solved: the controller works with the dual problem, mapping forces rather than velocities. As a result, it remains well-defined at singularities, and a target out of reach is harmless, as the arm simply stretches towards it.

You will develop a simple Python library for the geometric wrench control of a serial-chain robot step by step: you will describe the kinematics of the robot, compute its forward kinematics and geometric Jacobian, compute the virtual wrench, and map it to the joint torques. You will then test the controller on the UR5e arm in MuJoCo, first on a static target and then on a moving one.

⚠**Warning on the use of AI tools.** We are well aware that today's AI models, capable of solving open mathematical problems, can effortlessly complete this assignment. We nevertheless urge you to make your own effort to understand and solve the assignment yourself. While we are strictly against using AI tools to generate the solutions instead of you, we welcome their use for discussion: asking 'why' questions, asking to explain the choices made in the code we provide, asking for guidance on Python syntax, etc. In your prompts, you may explicitly instruct the model not to give you a solution, but to help you understand the problem. What you learn by completing this assignment will greatly help you in the final exam.

2 Implementation

2.1 Installation

Everything needed for the assignment will be installed inside a self-contained `conda` environment – an isolated space containing a specific version of Python and all the required packages, not interfering with what is already installed on your machine. To set up the assignment, follow these steps:

1. **Install an IDE (optional).** You will be editing Python files and running them from a terminal. An IDE (integrated development environment) is a convenient tool that simplifies code development, especially for large projects. Both of the following widely-used IDEs are free, and either is a good choice:

   - **Visual Studio Code**, https://code.visualstudio.com. After installing it, add the *Python* extension by Microsoft (the Extensions panel on the left).
   - **PyCharm**, https://www.jetbrains.com/pycharm/.

   The commands below are typed into a terminal, and it is convenient to use the one built into the IDE. A plain system terminal works just as well if you prefer it.

2. **Get the project and open it in the IDE.** Go to https://github.com/vasilysent/robotics_course, click the green *Code* button and then *Download ZIP*. Unzip the downloaded file and copy the `assignment` folder to a convenient location.

Now open the `assignment` *folder* with *File* > *Open Folder* in VS Code or *File* > *Open* in PyCharm. Then open a terminal inside the IDE: *Terminal* > *New Terminal* in VS Code, or the *Terminal* tab at the bottom of the PyCharm window. That terminal starts in the project folder. Verify that it is the right one by listing the contents with `ls` (macOS, Linux) or `dir` (Windows): the listing must contain `environment.yml`. If it does not, go to the assignment folder with `cd path/to/the/assignment`.

3. **Install conda package manager.** `conda` is the package manager that will build the environment. Check whether you already have it:

   ```
   conda --version
   ```

   If this prints a version number, such as `conda 26.3.2`, continue with step 4. If instead it reports that the command is not found or not recognised, install **Miniforge**, a minimal `conda` installer that comes set up with the **conda-forge** package channel — the channel that the `environment.yml` of this project asks for. Download it and follow the instructions for your operating system given in its README at

   https://github.com/conda-forge/miniforge

   When the installation has finished, **close the terminal and open a new one**, and check `conda --version` again. If the command is still not found, the terminal does not know about `conda` yet:

   - **macOS and Linux:** run `conda init zsh` (macOS) or `conda init bash` (Linux) from the folder in which Miniforge was installed, then close the terminal and open a new one.
   - **Windows:** the installer deliberately leaves `conda` off the search path, so only one shell knows about it: **Miniforge Prompt**, from the Start menu. (If you installed Anaconda or Miniconda instead, the equivalent is called *Anaconda Prompt*.) To teach the terminal of the IDE about `conda` as well:

     (a) Open **Miniforge Prompt** and run

     ```
     conda init powershell
     ```

     This writes a PowerShell startup file that makes `conda` available. Run also `conda init cmd.exe` if you prefer the Command Prompt.

     (b) Open a new terminal in the IDE and run

     ```
     Set-ExecutionPolicy RemoteSigned -Scope CurrentUser
     ```

     answering `Y` if prompted.

     (c) **Restart the IDE.** The terminal now recognises `conda`.

4. **Create the environment.** In the terminal, in the project folder:

   ```
   conda env create -f environment.yml
   ```

This creates an environment named `mdk-assignment` with Python 3.11 and installs the project into it together with everything it requires: MuJoCo, NumPy, PyYAML, imageio and matplotlib. It downloads a few hundred megabytes and takes a few minutes. It has to be done only once.

If it fails with `EnvironmentFileNotFound`, you are not in the project folder. Go there with `cd path/to/the/project`.

5. **Activate the environment.**

   ```
   conda activate mdk-assignment
   ```

   The command prompt now begins with `(mdk-assignment)`. That prefix is how you can recognise that the environment is active.

6. **Check that it worked.** First verify that MuJoCo is available:

   ```
   python -c "import mujoco; print(mujoco.__version__)"
   ```

   This should print `3.10.0`. Then run the first script[1]:

   ```
   python scripts/01_view_robot.py       (Linux, Windows)
   mjpython scripts/01_view_robot.py     (macOS)
   ```

   An interactive viewer should open showing the UR5e arm in its reference ($q = 0$) configuration. In the 'Control' panel on the right, you can move the sliders to change the joint angles and see the robot move.

7. **Look at the project structure.** The project follows the standard *src layout* of a Python project: the code lives in the folder `src/assignment`, which makes `assignment` an installable library package, described by `pyproject.toml` and `requirements.txt`. The environment you created installs this package, so the scripts can import it with `import assignment` from anywhere. The library code, the scripts that run it, and the simulation models are kept in separate folders:

   ```
   models/                  MuJoCo models of the UR5e
   scripts/                 the numbered scripts you run
   src/assignment/          the library code provided
   src/assignment/student/  the files you complete
   ```

   Everything you need to write is in the `student` folder: the file `ur5e_kinematics.yaml` with the robot description, the Lie group functions in `lie_groups.py`, the kinematics in `kinematics.py`, and the controller in `control.py`. The settings of the controller, such as the target and the gains, are in `config.py` in the same folder. The rest of the project is provided and does not need to be changed. Since the project is installed in editable mode, your changes to these files take effect the next time you run a script.

When you come back to the project, you may need to reactivate the environment. If the command prompt does not begin with `(mdk-assignment)`, run

[1] On macOS, the scripts that open the viewer must be run with `mjpython` rather than `python`.

```
conda activate mdk-assignment
```

from the project folder.

Troubleshooting:

- *`ModuleNotFoundError: No module named 'mujoco'` or `'assignment'`.* The environment is not active; see step 5.
- *On macOS, the script exits with* `On macOS, run this script with: mjpython ...` Do exactly that, as in step 6.
- *On Windows, the terminal of the IDE does not recognise* **`conda`***, although* ***Miniforge Prompt*** *does.* Follow the three points for Windows in step 3. If the IDE opens PowerShell 7 (`pwsh`) rather than Windows PowerShell, run `conda init powershell` from inside `pwsh` too in step 3(a).
- *The IDE underlines the imports in red, or its Run button fails, while the same script works in the terminal.* The IDE is using a different Python interpreter. In VS Code, press `Ctrl+Shift+P`, run *Python: Select Interpreter* and choose `mdk-assignment`. In PyCharm, go to *Settings* > *Project* > *Python Interpreter* > *Add* > *Conda Environment*, and select the existing `mdk-assignment`.
- *`EnvironmentFileNotFound: 'environment.yml' file not found`.* You are in the wrong folder; see step 4.
- *Importing* **`assignment`** *fails after you have moved or renamed the project folder.* Run '`pip install -e .`' from the project folder, with the environment active.

2.2 Robot description

In this step, you will assign a frame to each link of the robot and specify the unit twist of each joint in the reference ($q = 0$) configuration.

Modern robotics libraries usually split information about the robot into two parts: *Model* – constant description of the robot (the kinematic tree, joint types and placements, link inertias, etc.), and *Data* – the configuration-dependent information (current link poses, the Jacobian, the mass matrix). In this project, we will develop a simplified version of such a library: in parallel to MuJoCo's `MjModel` and `MjData` we will use our own classes `ChainModel` and `ChainData`.

The Model of a robot is built from a robot description file; common formats of such files are URDF and MJCF. In this project, we will use our own simplified format for serial-chain mechanisms. We agree to number the links (base to tip) from 0 to n and the joints from 1 to n, so that joint i connects link $i-1$ (its parent) to link i (its child). Our robot description file is a YAML file containing the relative poses $H_i^{i-1}(0)$ of the adjacent links in the reference ($q = 0$) configuration and the joint unit twists $\hat{T}_i^{i,(i-1)}$ expressed in the child link frames for $i \in \{1, \ldots, n\}$. We use the child (not parent) link frames to express the unit twists because it is a prevailing convention in common robotics libraries. The file additionally contains the

relative pose H_0^{w} of the base link frame Ψ_0 to the world frame Ψ_{w} and the relative pose H_{ee}^n of the dedicated end-effector frame Ψ_{ee} to the frame Ψ_n.

MuJoCo's MJCF robot description for the UR5e robot is given in

`models/universal_robots_ur5e/ur5e.xml`

and is used for the simulation, while our robot description is given in

`src/assignment/student/ur5e_kinematics.yaml`

Your task is to fill the latter file with the correct data. The robot's dimensions *in millimeters* are provided in Figure 1. The end-effector frame Ψ_{ee} is fixed to the final link (as shown[2] with

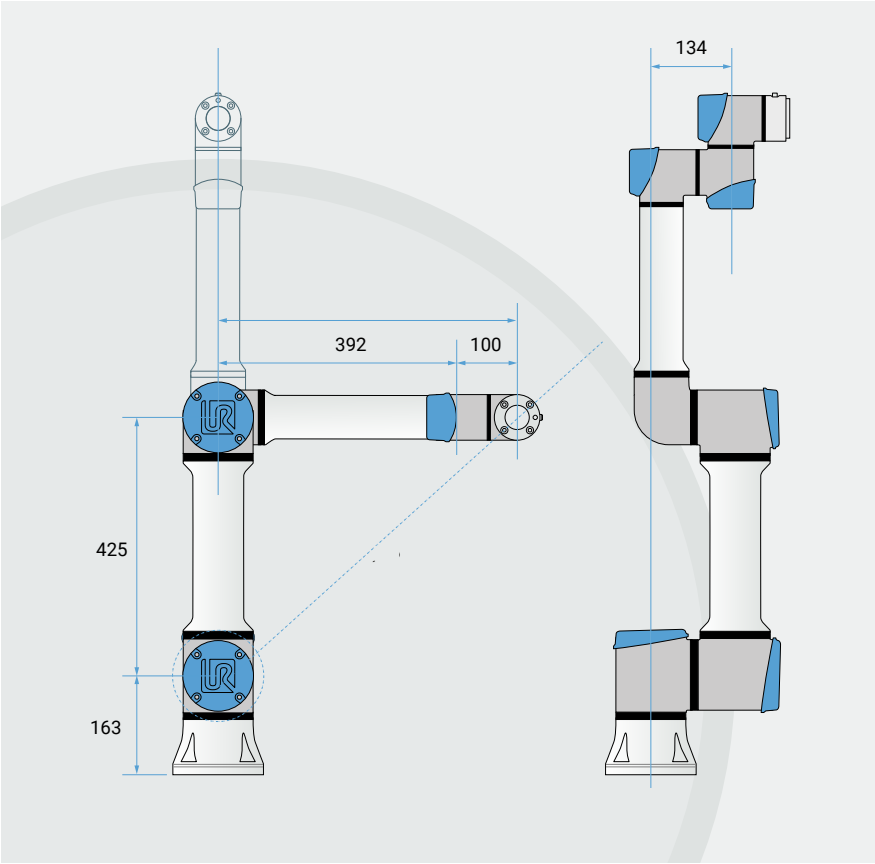


Figure 1: UR5e robot arm dimensions in millimeters.

`01_view_robot.py`), its z-axis is aligned with axis q^6 and points outwards, axis q^5 lies in its negative-z half-space a distance of 100 mm from its xy-plane. At the $q = 0$ configuration, the y-axis of Ψ_{ee} points vertically upwards. It is important that the end-effector frame you assign matches this description with both the position and the orientation.

The positive direction of a joint rotation can be established by using the control sliders in the viewer when running `01_view_robot.py`.

[2]We use the standard convention for labelling frames' axes: RGB = xyz, that is, the red arrow corresponds to the x-axis, the green one to the y-axis and the blue – to the z-axis.

Note that expressions for unit twists take a simple form if the origin of the frame in which they are expressed is placed at the joint axis, it is therefore convenient to place the frame Ψ_i at the joint i – a simple mnemonic with our child frame convention. It does not matter where exactly the frame is placed along the joint axis.

To verify the frames' placement run the second script with

```
python scripts/02_check_frames_reference.py       (Linux, Windows)
mjpython scripts/02_check_frames_reference.py     (macOS)
```

Note that this script draws the specified frames just in the reference configuration, and they stay fixed even if the robot is moved using the sliders.

2.3 Forward kinematics and Jacobian

This step is the central step in the assignment. You will implement a function computing the forward kinematics and the geometric Jacobian of the robot.

The class `ChainData`, defined in

```
src/assignment/student/kinematics.py
```

stores the configuration-dependent data of the robot, in particular, the current relative poses $H_i^{i-1}(q)$ of adjacent links, and the Jacobian $J^{\text{ee}}(q)$ expressed in the end-effector frame Ψ_{ee}. Your task is to complete the function `update_kinematics` that, given the current joint configuration q, updates all the kinematic data of the robot in `ChainData`.

The kinematic relation for a unit twist expressed in the child link frame is

$$H_i^{i-1}(q) = H_i^{i-1}(0)\exp(\widetilde{\hat{T}}_i^{i,(i-1)} q^i). \tag{3}$$

The geometric Jacobian in the end-effector frame is a $6 \times n$ matrix whose i-th column is the unit twist $\hat{T}_i^{\text{ee},(i-1)}$ of joint i expressed in the end-effector frame Ψ_{ee}.

Split your code into separate elementary functions. It is crucial to implement the update function efficiently, as it will be called at every control step during the tracking: avoid repeatedly computing the same intermediate results and be wise by employing the known structure of the Lie group SE(3) for computing exponentials and inverses instead of using the generic matrix operations. For example, implement a function `so3_exp` for the exponential map of $\mathfrak{so}(3)$ using the Rodrigues' formula and put it in `lie_groups.py`, as it is a general Lie group function that can be reused in other projects. The file `lie_groups.py` already declares the functions `unskew`, `so3_exp`, `se3_exp` and `se3_inverse`; keep their names, since the provided code imports some of them, and add any other functions you need.

To verify your implementation of the forward kinematics, run the third script with

```
python scripts/03_check_forward_kinematics.py       (Linux, Windows)
mjpython scripts/03_check_forward_kinematics.py     (macOS)
```

In this script, the frames are visualised in the current configuration, according to your update function.

2.4 Virtual wrench

In this step, you will implement the wrench that pulls the end-effector towards a desired frame Ψ_d as a virtual spring and damper would. The functions of this and the next step are in `src/assignment/student/control.py`.

Spring. The spring acts on the relative configuration of the end-effector frame with respect to the desired frame,

$$H^\mathrm{d}_\mathrm{ee} = \left(H^\mathrm{w}_\mathrm{d}\right)^{-1} H^\mathrm{w}_\mathrm{ee} = \begin{pmatrix} R^\mathrm{d}_\mathrm{ee} & p^\mathrm{d}_\mathrm{ee} \\ 0 & 1 \end{pmatrix}. \tag{4}$$

The wrench of a spatial spring connecting the two frames, expressed in the end-effector frame, is[3]

$$\begin{aligned} \tilde{m} &= -2\,\mathrm{as}\big(G_o R^\mathrm{d}_\mathrm{ee}\big) - \mathrm{as}\big(G_t R^\mathrm{ee}_\mathrm{d} \tilde{p}^\mathrm{d}_\mathrm{ee} \tilde{p}^\mathrm{d}_\mathrm{ee} R^\mathrm{d}_\mathrm{ee}\big) - 2\,\mathrm{as}\big(G_c \tilde{p}^\mathrm{d}_\mathrm{ee} R^\mathrm{d}_\mathrm{ee}\big), \\ \tilde{f} &= -R^\mathrm{ee}_\mathrm{d}\,\mathrm{as}\big(G_t \tilde{p}^\mathrm{d}_\mathrm{ee}\big) R^\mathrm{d}_\mathrm{ee} - \mathrm{as}\big(G_t R^\mathrm{ee}_\mathrm{d} \tilde{p}^\mathrm{d}_\mathrm{ee} R^\mathrm{d}_\mathrm{ee}\big) - 2\,\mathrm{as}\big(G_c R^\mathrm{d}_\mathrm{ee}\big), \end{aligned} \tag{5}$$

where $\mathrm{as}(A) = \frac{1}{2}(A - A^\top)$ is the skew-symmetric part of a matrix, $R^\mathrm{ee}_\mathrm{d} = (R^\mathrm{d}_\mathrm{ee})^\top$, and $\tilde{m}$ and $\tilde{f}$ are the skew-symmetric matrix forms of the moment m and the force f of the wrench $W^\mathrm{ee}_\mathrm{s} = \begin{pmatrix} m & f \end{pmatrix}$. The matrices G_o, G_t and G_c are the orientational, translational and coupling co-stiffness matrices. We use an isotropic spring, for which they are

$$G_o = \tfrac{k_o}{2} I, \qquad G_t = \tfrac{k_t}{2} I, \qquad G_c = \tfrac{k_c}{2} I, \tag{6}$$

with the rotational, translational and coupling stiffnesses k_o, k_t and k_c. The wrench vanishes when $H^\mathrm{d}_\mathrm{ee} = I$, that is, when the end-effector has reached the desired configuration.

Implement (5) in the function `spring_wrench`.

Saturation. The force of the spring grows with the distance to the target, so, if left unconstrained, a distant target would result in joint torques that the motors cannot deliver. The actuators then clip each joint torque separately (according to their torque limits), which distorts the wrench direction. It is better to limit the wrench before it is mapped to the joint torques, scaling the moment and the force separately, since they are measured in different units:

$$m \leftarrow m \min\Big(1, \frac{m_\mathrm{max}}{\|m\|}\Big), \qquad f \leftarrow f \min\Big(1, \frac{f_\mathrm{max}}{\|f\|}\Big). \tag{7}$$

Scaling a vector does not change its direction, so the capped spring still pulls towards the target. Implement this in the function `clip_wrench`.

Damping. A spring alone makes the end-effector oscillate around the target, so a damper is added that opposes the motion of the end-effector *relative to the target*. The relative twist expressed in the end-effector frame is

$$T^\mathrm{ee,d}_\mathrm{ee} = T^\mathrm{ee,w}_\mathrm{ee} - \mathrm{Ad}_{H^\mathrm{ee}_\mathrm{d}}\, T^\mathrm{d,w}_\mathrm{d}, \qquad T^\mathrm{ee,w}_\mathrm{ee} = J^\mathrm{ee}(q)\,\dot{q}, \tag{8}$$

[3]This formula is derived from a potential function on SE(3); see S. Stramigioli, *Modeling and IPC Control of Interactive Mechanical Systems — A Coordinate-Free Approach*, Lecture Notes in Control and Information Sciences, vol. 266, Springer London, 2001, Section 5.4.

where $T_{\mathrm{d}}^{\mathrm{d,w}}$ is the twist of the desired (target) frame, zero for a target standing still.

The control wrench is the capped spring wrench plus the damping wrench[4],

$$\big(W^{\mathrm{ee}}\big)^\top = \mathrm{clip}\big(W_{\mathrm{s}}^{\mathrm{ee}}\big)^\top - D\,T_{\mathrm{ee}}^{\mathrm{ee,d}}, \qquad D = \begin{pmatrix} d_o I & 0 \\ 0 & d_t I \end{pmatrix}, \tag{9}$$

with the rotational and translational damping coefficients d_o and d_t. Only the spring is capped, not the damper. Implement (9) in the function `control_wrench`.

2.5 Control torques

In this step, you will map the control wrench to the joint torques commanded to the motors and run the target tracking simulation scripts.

The action of the dual of the Jacobian (2) in the end-effector frame may be written as:

$$\tau^\top = J^{\mathrm{ee}}(q)^\top \big(W^{\mathrm{ee}}\big)^\top. \tag{10}$$

Unlike its inverse, the transpose of the Jacobian always exists, also at singular configurations.

The dynamics of the robot is then governed by the equation

$$M(q)\,\ddot{q} + C(q,\dot{q})\,\dot{q} = J^{\mathrm{ee}}(q)^\top \big(W^{\mathrm{ee}}\big)^\top. \tag{11}$$

The gravity term is not present in the equation, since it is compensated for in the simulation[5], so that the robot behaves as if it were in weightless.

Implement (10) in the function `control_torque`.

The function is called in the method `step` of the class `TargetTracker` in `control.py`. Read it: at every simulation step it updates the kinematic data at the current configuration, computes the twist of the end-effector and the control wrench, and commands the resulting joint torques[6]. The simulation uses a model of the UR5e with torque-controlled motors,

```
models/universal_robots_ur5e/scene_torque.xml
```

in which gravity compensation is enabled.

To test your implementation, run the fourth script with

```
python scripts/04_track_static_target.py       (Linux, Windows)
mjpython scripts/04_track_static_target.py     (macOS)
```

[4] In this description, we represent wrenches W and joint torques τ as row covectors and transpose them explicitly when necessary; in the code, both vectors and covectors are represented by 1d arrays without such distinction.

[5] This matches the direct torque control on a real UR robot, where only the control torques are commanded, and the gravity is compensated for automatically.

[6] Before passing the control torques to the simulation, the joint speed guard is applied, which prevents the robot from approaching its joint speed limits. This is safety measure is particularly important near singular configurations, where the task space damping does not prevent the joints from moving too fast.

The end-effector should move to the target frame and stay there. Note that the simulation may run slower than real time on a slow machine. The target, the stiffnesses, the damping coefficients and the limits of the wrench are set in `src/assignment/student/config.py`. Try moving the target out of the reach of the robot: since the controller never inverts the Jacobian, the arm simply stretches towards the target and remains stable.

To see the motion to the static target in true real time, record a video with the fifth script[7]:

```
python scripts/05_record_static_target.py
```

The video and the error plot together with the used `config.py` are saved in the `runs/` folder in a timestamped subfolder.

Finally, you can test the controller on a moving target following a figure-eight trajectory with a fixed orientation. Run the sixth script with

```
python scripts/06_record_figure_eight.py
```

to record a video and build an error plot of the moving target tracking. The trajectory settings can be adjusted in `config.py`.

3 Submission

Copy the folder `src/assignment/student` into another location, **rename it to your student number** and compress it into a `.zip` archive. Submit the archive to Canvas.

[7]Since the viewer is not opened, the script is run with `python` rather than `mjpython` also on macOS